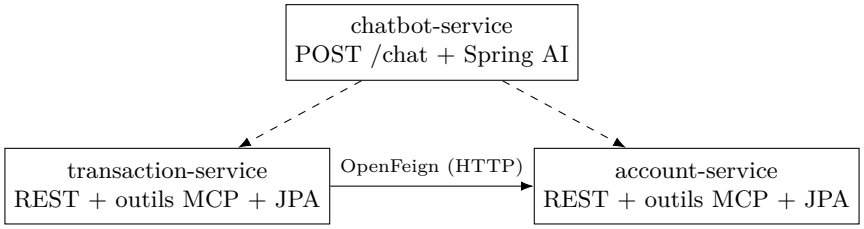
Épreuve technique de sélection

Rapport d'avancement — plateforme bancaire

Youssef Rahioui | 30 octobre 2026

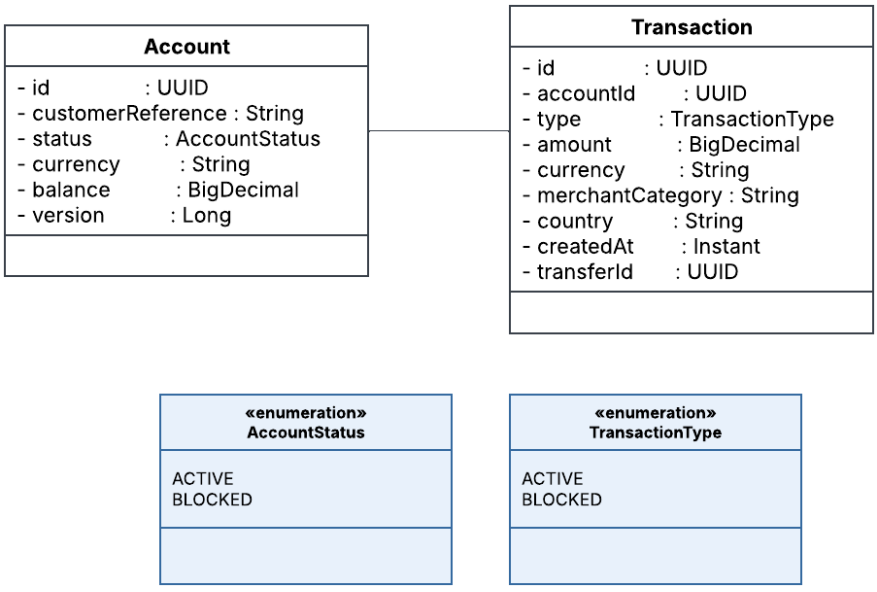
1. Architecture du projet

Le dépôt Maven `bank` comprend deux microservices métier, `account-service` et `transaction-service`, ainsi qu'un `chatbot-service`. Les services métier communiquent par OpenFeign ; le chatbot dispose d'un client MCP configuré pour les interroger.



Pointillés : connexions MCP configurées ; appel réel aux outils par le LLM non vérifié.

Diagramme de classes (fourni avec le projet). Le lien entre les entités est logique, par `accountId`, et non une association JPA.



Note : dans le code, `TransactionType` vaut `DEBIT/CREDIT` (et non `ACTIVE/BLOCKED` comme sur l'image). La liaison dessinée n'est pas une association JPA. Le solde est en `BigDecimal` dans le code.

2. Réalisation et vérification

account-service. Entité `Account` (UUID, référence client, statut, devise, solde en `BigDecimal`, version), dépôt JPA, service et contrôleur REST. Création et consultation de compte, consultation du solde, débit et crédit. Les mouvements contrôlent le montant positif, le statut actif, la devise et, pour un débit, les fonds disponibles.

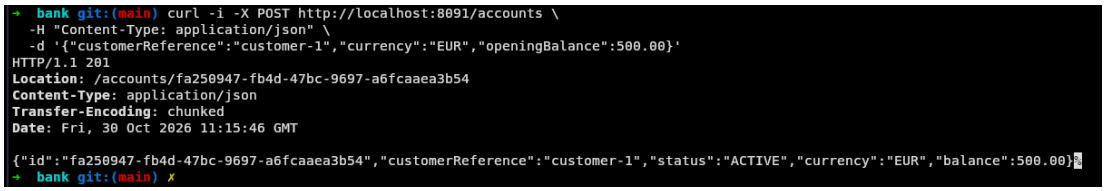
transaction-service. Entité `Transaction`, dépôt JPA, service et contrôleur REST. Création, consultation, liste par compte et virement ; contrôles du compte, du solde, de la devise et du plafond local fixé à 1000 (`bank.transaction.max-amount`). `AccountClient` communique avec `account-service` via Feign pour consulter, débiter ou créditer. Les deux services organisent le code en `model/`, `repository/`, `service/` et `web/rest/`.

Outils MCP ajoutés. Les classes `AccountTools` et `TransactionTools` appellent les mêmes services métier que les contrôleurs REST. Elles exposent la consultation du compte et du solde, ainsi que la consultation et la création de transactions.

chatbot-service. Le contrôleur expose `POST /chat` avec un message et renvoie une réponse. `ChatService` transmet le message au `ChatClient` Spring AI ; `ChatConfiguration` lui associe les outils du client MCP. Le modèle configuré est `gpt-4o-mini` et la clé provient de `OPENAI_API_KEY`. L'appel réel au modèle et aux serveurs MCP n'est pas démontré par les tests présents.

Tests exécutés. `mvn test -q` passe depuis la racine : tests métier/REST dans les deux services, tests du service et du contrôleur du chatbot avec dépendances simulées. Aucun essai LLM/MCP de bout en bout.

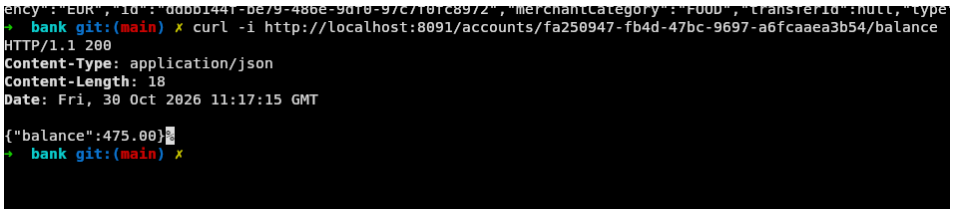
Captures des essais REST (fournies avec le projet). Un compte EUR est créé avec 500, puis une transaction de 25 est enregistrée. La consultation du solde retourne 475 ; ces réponses illustrent le débit entre les deux services.



1. Création du compte : `POST /accounts`, HTTP 201.



2. Création de la transaction : `POST /transactions`, HTTP 201.



3. Vérification du solde : `GET /accounts/{id}/balance`, HTTP 200.